Machine Coding: Todo App

- **Problem:** Build a todo app: add a todo (with validation), mark it complete/active, edit it, delete it, search by text, filter by All / Active / Completed, show how many are left, clear completed, and keep the todos after a page refresh.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Assumptions written down
Use cases + architecture + state	8 min	Table, component tree, state shape
Core build (add, list, toggle, delete)	20 min	Working todo list
Edit, search, filter, counts, persistence	15 min	All features working
Edge cases, a11y, polish	5 min	Empty states, labels
Test manually + wrap-up	7 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I won't store the filtered list in state. I'll derive it from todos + filter + search, so it can never go out of sync."* If you run out of time, a clear plan you talked through still scores well.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Add a todo; reject empty, too-short (< 4 chars), too-long (> 100 chars) and duplicate titles, with an inline error.
- Toggle a todo between Active and Completed.
- Edit a todo inline (Save / Cancel / Escape), with the same validation as add.
- Delete a todo.
- Search by title (live, case-insensitive).
- Filter: All / Active / Completed. Search and filter work **together**.
- Show "N items left" and a "Clear completed" button.
- Persist todos in localStorage so they survive a refresh.

Non-functional requirements

Concern	What it means here
Performance	Filtering a few hundred items on each render is cheap. No `useMemo` or debounce until a measurement says so.
Scalability	All todo updates go through one reducer, so new actions (reorder, priority) are one new `case`.
Error handling	Validation errors inline; corrupted or full localStorage must not crash the app.
Accessibility	Every input has a label, the checkbox is wrapped in its `<label>`, filter buttons use `aria-pressed`, errors use `role="alert"`, the count uses `aria-live`.
Reusability	`validateTitle` is one pure function used by both add and edit.
Extensibility	Swapping localStorage for an API only touches `loadTodos` / `saveTodos` (and the dispatch calls).

Clarifying questions to ask the interviewer

1. What are the validation rules: minimum/maximum length, and are duplicate titles allowed?
2. Do we need edit? Inline, or a modal? What happens if the edited text is empty: error, or delete the todo?
3. Should search and filter combine (e.g. "milk" inside Completed)?
4. Should todos persist across refresh? localStorage, or is there an API?
5. Any ordering rule: newest first, or in the order they were added?
6. Is drag-and-drop reordering or a due date needed? (Usually: no, mention it as an extension.)

Assumptions we will build with

- Title is trimmed; 4-100 characters; duplicates (case-insensitive) are rejected.
- Inline edit: Enter or Save saves, Escape or Cancel cancels; an empty edit shows an error (it does not delete).

- Search and filter combine. Search ignores case and surrounding spaces.
- Todos are shown in the order they were added (oldest at the top).
- localStorage only, no backend.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type "Buy milk" + Enter	Todo added at the bottom, input cleared, "1 item left"
Submit " " or "abc"	Inline error ("cannot be empty" / "at least 4 characters"); nothing added
Submit " buy MILK " when "Buy milk" exists	"This todo already exists"
Type after an error	Error clears
Click a todo's checkbox	Title struck through; count updates
Toggle a todo while the filter is "Active"	It disappears from the list immediately
Click Edit (or double-click the title)	Row becomes an input with the current title, focused
Edit + Enter / Save	Title updated (trimmed); validated like add, but it may keep its own title
Edit + Escape / Cancel	Back to the original title, nothing saved
Click Delete	Todo removed; count updates
Type in search	List narrows live; works together with the active filter
Click All / Active / Completed	List filtered; the selected button is highlighted (`aria-pressed`)
Click "Clear completed"	All completed todos removed; button disabled when there are none
Refresh the page	Same todos, same completed state
No todos at all	"No todos yet. Add your first one above." and no footer
Todos exist but none match	`No todos match "xyz".` or `No completed todos.`

3. Design the architecture

Module	Responsibility
`App`	Owns `todos` (reducer), `filter` , `search` ; derives the visible list and counts; renders everything
`todoReducer.js`	Pure reducer (add / toggle / edit / delete / clearCompleted), `validateTitle` , `loadTodos` / `saveTodos`
`TodoForm`	Input + Add button; owns its draft text and error
`TodoToolbar`	Search input + filter buttons (stateless, controlled by `App`)
`TodoItem`	One row: checkbox, title, Edit, Delete; owns its edit mode, draft and error

```
App  (todos via useReducer, filter, search)
├── TodoForm        → onAdd(title), validate(title)
├── TodoToolbar     → search/onSearch, filter/onFilter
├── <ul> visibleTodos.map
│    └── TodoItem   → onToggle(id), onEdit(id, title), onDelete(id), validate(title, id)
└── footer          → "N items left", Clear completed
todoReducer.js  ← pure logic, imported by App
```

Start simple. First version: everything in `App` with `useState`. Move to a reducer once you have 4+ kinds of updates, and pull out `TodoItem` once a row has its own state (edit mode).

4. Design the state / data model

State	Where	Purpose
`todos`	App (`useReducer`)	The source of truth: list of todo objects
`filter`	App	`"all" | "active" | "completed"`
`search`	App	Raw search text (controlled input)
`text` , `error`	TodoForm	Draft of the new todo + its validation error
`isEditing` , `draft` , `error`	TodoItem	Inline edit mode for that one row

```
// App
const [todos, dispatch] = useReducer(todoReducer, undefined, loadTodos); // lazy init from storage
const [filter, setFilter] = useState("all"); // all | active | completed
const [search, setSearch] = useState("");

// derived on every render
const query = search.trim().toLowerCase();
const visibleTodos = todos.filter(
  (t) => t.title.toLowerCase().includes(query) &&
         (filter === "all" || (filter === "active" ? !t.completed : t.completed))
);
const activeCount = todos.filter((t) => !t.completed).length;
const completedCount = todos.length - activeCount;
```

Senior note

- **Derived, not stored:** `visibleTodos` , `activeCount` , `completedCount` , and the empty-state message. A `filteredTodos` state would need updating in 5 places (add, toggle, edit, delete, filter change), and one miss is a bug.
- **Ownership:** `todos` , `filter` and `search` live in `App` because several children need them. Draft text and edit mode are local to `TodoForm` / `TodoItem` : nobody else needs them, and keeping them local means typing in one row doesn't re-render the whole app.

- **Status is one boolean `completed`,** not a string `"Active"/"Completed"`: you can't make a typo in a boolean.

5. Design the data structures and contracts

Todo object (stable `id` from `crypto.randomUUID()`: `Date.now()` can collide when two todos are added in the same millisecond, and an array index changes when items are deleted):

```
{ "id": "3f2a9c1e-…", "title": "Buy milk", "completed": false, "createdAt": 1793550000000 }
```

Reducer actions (the internal "API")

Action	Payload	Result
`add`	`{ title }`	New todo appended (title trimmed, `completed: false`)
`toggle`	`{ id }`	That todo's `completed` flipped
`edit`	`{ id, title }`	That todo's title replaced (trimmed)
`delete`	`{ id }`	That todo removed
`clearCompleted`	-	All completed todos removed

Validation contract: `validateTitle(title, todos, ignoreId?) → ""` when valid, or an error message. `ignoreId` lets an edited todo keep its own title without counting as a duplicate.

Storage contract (mocked backend): `loadTodos() → todo[]` (returns `[]` for missing or corrupted data), `saveTodos(todo[])` (never throws). With a real API these become `GET /todos` and `PUT/PATCH /todos/:id`.

6. Trace the data flow

Toggle a todo while the filter is "Active":

```
User clicks the checkbox of "Buy milk"
  ↓ TodoItem: onToggle(todo.id)
  ↓ App: dispatch({ type: "toggle", id })
  ↓ todoReducer returns a NEW array; only that todo is a new object with completed: true
  ↓ React re-renders App
  ↓ derived: visibleTodos (filter "active") no longer contains "Buy milk"; activeCount -1;
completedCount +1
  ↓ effect: saveTodos(todos) writes to localStorage
  ↓ UI: row disappears, "1 item left", "Clear completed (1)" enabled
```

7. Important technical challenges

Challenge 1: stable ids and keys

- *Problem:* The list uses `key={index}`. You open edit on row 2 ("Walk dog"), then delete row 1. Row 2 becomes index 0, so React gives its edit state (`isEditing`, `draft`) to the wrong todo. A `<>` fragment without a key around each row also triggers warnings.
- *Solution:* Give every todo a unique id when it's created and use it as the key and for every update.

```
case "add":
  return [...todos, { id: crypto.randomUUID(), title: action.title.trim(), completed: false, createdAt:
Date.now() }];
// render
{visibleTodos.map((todo) => <TodoItem key={todo.id} todo={todo} ... />)}
```

Challenge 2: immutable updates in one place

- *Problem:* `todo.completed = true; setTodos(todos)` mutates the same array, so React sees the same reference and doesn't re-render. Update logic scattered across 5 handlers is also easy to get wrong.
- *Solution:* A pure reducer that always returns new arrays/objects. It's easy to test without any UI.

```
case "toggle":
  return todos.map((t) => (t.id === action.id ? { ...t, completed: !t.completed } : t));
case "clearCompleted":
  return todos.filter((t) => !t.completed);
```

Challenge 3: search + filter + edit validation together

- *Problem:* (a) Search "milk" with filter "Completed" must apply both rules. (b) Editing "Buy milk" to "Buy milk " must not say "already exists" just because it matches itself.
- *Solution:* Derive the list with both conditions, and use one validator with an `ignoreId`.

```
const duplicate = todos.some((t) => t.id !== ignoreId && t.title.toLowerCase() === value.toLowerCase());
```

Challenge 4: persistence without crashes

- *Problem:* Someone has `todos = "{broken"` in localStorage, or storage is blocked in a private window: `JSON.parse` throws and the whole app goes white. Reading storage on every render is also wasteful.
- *Solution:* Lazy initialiser (read once), `try/catch` on read and write, and a `useEffect` that saves whenever `todos` changes.

```
const [todos, dispatch] = useReducer(todoReducer, undefined, loadTodos);
useEffect(() => saveTodos(todos), [todos]);

export function loadTodos() {
  try {
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY));
    return Array.isArray(data) ? data : [];
  } catch {
    return [];
```

```
  }
}
```

8. Implement in the correct order

#	Step	Working result
1	`App` with a controlled input + Add, todos in `useState`, render `<ul>` with `key={todo.id}`	Can add and see todos
2	Validation: trim, min/max length, inline error	Bad input rejected
3	Toggle completed (checkbox + strike-through)	Can complete todos
4	Delete	Can remove todos
5	Move updates into `todoReducer` (add/toggle/delete)	Same behaviour, cleaner
6	Filter buttons (All/Active/Completed), derived list	Filtering works
7	Search input combined with filter	Search works
8	Footer: items left + Clear completed	Counts work
9	Extract `TodoItem`; add inline edit (Enter/Escape/Save/Cancel) + duplicate check with `ignoreId`	Editing works
10	Persistence: lazy load + save effect + try/catch	Survives refresh
11	Empty states, labels, `aria-pressed`, `role="alert"`, `aria-live`	Polished + accessible

9. Cover all states and edge cases

State / edge case	Handling
Initial (empty storage)	"No todos yet. Add your first one above.", no footer
Initial (saved todos)	Loaded once from localStorage
Adding with an invalid title	Inline error under the input, `aria-invalid`
Success (added)	Input cleared, error cleared, count updated
Whitespace-only / leading-trailing spaces	Trimmed before validating and saving
Duplicate title (any case)	"This todo already exists"
Very long title	Rejected over 100 characters; the input caps typing a little above that
Editing a row	Only that row changes to an input; others stay interactive
Edit to the same title / own title	No duplicate error (`ignoreId`); no dispatch if unchanged
Edit to empty	Error shown, edit stays open
Escape during edit	Changes discarded

State / edge case	Handling
Toggle while filtered	The item leaves/enters the visible list immediately
No search/filter matches	Specific empty message (`No todos match "x"` / `No completed todos.`)
Clear completed with none completed	Button disabled
Delete the last todo	Back to the initial empty state
Corrupted / blocked localStorage	Starts with `[]`; saving failures are ignored
Same app open in two tabs	Last write wins (see trade-offs)

10. Final structure

```
_02_todo_app/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css            ← global reset
    ├── App.jsx              ← state owner: todos, filter, search; derived list + counts
    ├── App.css              ← component styles + .sr-only
    ├── todoReducer.js       ← reducer, validateTitle, loadTodos/saveTodos (pure, testable)
    └── components/
        ├── TodoForm.jsx     ← add input + error
        ├── TodoToolbar.jsx  ← search + filter buttons
        └── TodoItem.jsx     ← row: toggle, edit mode, delete
```

11. Full working code

`src/todoReducer.js`

```
const STORAGE_KEY = "todos";
export const MIN_LENGTH = 4;
export const MAX_LENGTH = 100;

export function loadTodos() {
  try {
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY));
    return Array.isArray(data) ? data : [];
  } catch {
    return []; // corrupted or blocked storage: start empty instead of crashing
  }
}

export function saveTodos(todos) {
  try {
```

```
    localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
  } catch {
    // storage full or blocked: the app keeps working in memory
  }
}

// Shared by "add" and "edit". Returns "" when valid. ignoreId lets an item keep its own title.
export function validateTitle(title, todos, ignoreId = null) {
  const value = title.trim();
  if (!value) return "Todo cannot be empty";
  if (value.length < MIN_LENGTH) return `Todo must be at least ${MIN_LENGTH} characters`;
  if (value.length > MAX_LENGTH) return `Todo must be at most ${MAX_LENGTH} characters`;
  const duplicate = todos.some((t) => t.id !== ignoreId && t.title.toLowerCase() ===
value.toLowerCase());
  return duplicate ? "This todo already exists" : "";
}

// Every update returns a new array: React only re-renders when the reference changes.
export function todoReducer(todos, action) {
  switch (action.type) {
    case "add":
      return [
        ...todos,
        { id: crypto.randomUUID(), title: action.title.trim(), completed: false, createdAt: Date.now()
},
      ];
    case "toggle":
      return todos.map((t) => (t.id === action.id ? { ...t, completed: !t.completed } : t));
    case "edit":
      return todos.map((t) => (t.id === action.id ? { ...t, title: action.title.trim() } : t));
    case "delete":
      return todos.filter((t) => t.id !== action.id);
    case "clearCompleted":
      return todos.filter((t) => !t.completed);
    default:
      return todos;
  }
}
```

src/App.jsx

```
import { useEffect, useReducer, useState } from "react";
import "./App.css";
import { loadTodos, saveTodos, todoReducer, validateTitle } from "./todoReducer";
import TodoForm from "./components/TodoForm";
import TodoToolbar from "./components/TodoToolbar";
import TodoItem from "./components/TodoItem";
```

```
function App() {
  const [todos, dispatch] = useReducer(todoReducer, undefined, loadTodos); // lazy init: read storage
once
  const [filter, setFilter] = useState("all"); // all | active | completed
  const [search, setSearch] = useState("");

  useEffect(() => saveTodos(todos), [todos]);

  // Derived, never stored: always in sync with todos, filter and search.
  const query = search.trim().toLowerCase();
  const visibleTodos = todos.filter(
    (t) =>
      t.title.toLowerCase().includes(query) &&
      (filter === "all" || (filter === "active" ? !t.completed : t.completed))
  );
  const activeCount = todos.filter((t) => !t.completed).length;
  const completedCount = todos.length - activeCount;

  const validate = (title, ignoreId) => validateTitle(title, todos, ignoreId);

  let emptyMessage = "";
  if (todos.length === 0) emptyMessage = "No todos yet. Add your first one above.";
  else if (visibleTodos.length === 0)
    emptyMessage = query ? `No todos match "${search.trim()}".` : `No ${filter} todos.`;

  return (
    <main className="container">
      <h1>Todo App</h1>

      <TodoForm onAdd={(title) => dispatch({ type: "add", title })} validate={validate} />
      <TodoToolbar search={search} onSearch={setSearch} filter={filter} onFilter={setFilter} />

      {emptyMessage ? (
        <p className="empty">{emptyMessage}</p>
      ) : (
        <ul className="todo-list">
          {visibleTodos.map((todo) => (
            <TodoItem
              key={todo.id}
              todo={todo}
              validate={validate}
              onToggle={(id) => dispatch({ type: "toggle", id })}
              onEdit={(id, title) => dispatch({ type: "edit", id, title })}
              onDelete={(id) => dispatch({ type: "delete", id })}
            />
          ))}
```

```
        </ul>
      )}

      {todos.length > 0 && (
        <footer className="footer">
          <span aria-live="polite">
            {activeCount} {activeCount === 1 ? "item" : "items"} left
          </span>
          <button
            type="button"
            className="secondary"
            onClick={() => dispatch({ type: "clearCompleted" })}
            disabled={completedCount === 0}
          >
            Clear completed ({completedCount})
          </button>
        </footer>
      )}
    </main>
  );
}

export default App;
```

src/components/TodoForm.jsx

```
import { useState } from "react";
import { MAX_LENGTH } from "../todoReducer";

// Owns only the draft text and its error; the parent decides what is valid and stores the todo.
const TodoForm = ({ onAdd, validate }) => {
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const err = validate(text);
    if (err) {
      setError(err);
      return;
    }
    onAdd(text);
    setText("");
    setError("");
  };

  return (
```

```
    <form className="todo-form" onSubmit={handleSubmit} noValidate>
      <label htmlFor="new-todo" className="sr-only">
        New todo
      </label>
      <input
        id="new-todo"
        type="text"
        placeholder="What needs to be done?"
        value={text}
        maxLength={MAX_LENGTH + 20}
        onChange={(e) => {
          setText(e.target.value);
          if (error) setError("");
        }}
        aria-invalid={Boolean(error)}
        aria-describedby={error ? "new-todo-error" : undefined}
        autoFocus
      />
      <button type="submit">+ Add</button>
      {error && (
        <p id="new-todo-error" role="alert" className="error">
          {error}
        </p>
      )}
    </form>
  );
};

export default TodoForm;
```

src/components/TodoToolbar.jsx

```
const FILTERS = ["all", "active", "completed"];

const TodoToolbar = ({ search, onSearch, filter, onFilter }) => (
  <div className="toolbar">
    <label htmlFor="search" className="sr-only">
      Search todos
    </label>
    <input
      id="search"
      type="search"
      placeholder="Search..."
      value={search}
      onChange={(e) => onSearch(e.target.value)}
    />
    <div className="filters" role="group" aria-label="Filter todos">
```

```
      {FILTERS.map((f) => (
        <button
          key={f}
          type="button"
          className={filter === f ? "active" : "secondary"}
          aria-pressed={filter === f}
          onClick={() => onFilter(f)}
        >
          {f[0].toUpperCase() + f.slice(1)}
        </button>
      ))}
    </div>
  </div>
);

export default TodoToolbar;
```

src/components/TodoItem.jsx

```
import { useState } from "react";

const TodoItem = ({ todo, onToggle, onDelete, onEdit, validate }) => {
  const [isEditing, setIsEditing] = useState(false);
  const [draft, setDraft] = useState(todo.title);
  const [error, setError] = useState("");

  const startEdit = () => {
    setDraft(todo.title);
    setError("");
    setIsEditing(true);
  };

  const cancelEdit = () => {
    setIsEditing(false);
    setError("");
  };

  const saveEdit = (e) => {
    e.preventDefault();
    const err = validate(draft, todo.id);
    if (err) {
      setError(err);
      return;
    }
    if (draft.trim() !== todo.title) onEdit(todo.id, draft);
    setIsEditing(false);
  };
```

```
  if (isEditing) {
    return (
      <li className="todo-item">
        <form className="edit-form" onSubmit={saveEdit}>
          <label htmlFor={`edit-${todo.id}`} className="sr-only">
            Edit {todo.title}
          </label>
          <input
            id={`edit-${todo.id}`}
            type="text"
            value={draft}
            onChange={(e) => setDraft(e.target.value)}
            onKeyDown={(e) => e.key === "Escape" && cancelEdit()}
            aria-invalid={Boolean(error)}
            aria-describedby={error ? `edit-${todo.id}-error` : undefined}
            autoFocus
          />
          <button type="submit">Save</button>
          <button type="button" className="secondary" onClick={cancelEdit}>
            Cancel
          </button>
          {error && (
            <p id={`edit-${todo.id}-error`} role="alert" className="error">
              {error}
            </p>
          )}
        </form>
      </li>
    );
  }

  return (
    <li className="todo-item">
      <label className={`todo-title ${todo.completed ? "done" : ""}`}>
        <input type="checkbox" checked={todo.completed} onChange={() => onToggle(todo.id)} />
        <span onDoubleClick={startEdit}>{todo.title}</span>
      </label>
      <button type="button" className="secondary" onClick={startEdit} aria-label={`Edit ${todo.title}`}>
        Edit
      </button>
      <button type="button" className="danger" onClick={() => onDelete(todo.id)} aria-label={`Delete
${todo.title}`}>
        Delete
      </button>
    </li>
  );
```

```
};

export default TodoItem;
```

src/App.css

```
.container { max-width: 640px; margin: 40px auto; padding: 20px; background: white; border-radius: 8px; font-family: system-ui, sans-serif; }
h1 { text-align: center; margin-bottom: 16px; }
input[type="text"], input[type="search"] { flex: 1; padding: 8px 10px; border: 1px solid #ccc; border-radius: 6px; font-size: 16px; }
input[aria-invalid="true"] { border-color: #dc2626; }
button { padding: 8px 14px; border: none; border-radius: 6px; background: #4f46e5; color: white; cursor: pointer; font-size: 14px; }
button:hover { background: #4338ca; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button.secondary { background: #e5e7eb; color: #111827; }
button.danger { background: #dc2626; }
.todo-form, .edit-form, .toolbar { display: flex; flex-wrap: wrap; gap: 8px; }
.toolbar { margin-top: 16px; }
.filters { display: flex; gap: 4px; }
.error { width: 100%; color: #dc2626; font-size: 14px; }
.todo-list { list-style: none; margin-top: 16px; }
.todo-item { display: flex; align-items: center; gap: 8px; padding: 8px 0; border-bottom: 1px solid #eee; }
.todo-item .edit-form { flex: 1; }
.todo-title { flex: 1; display: flex; gap: 8px; align-items: center; cursor: pointer; }
.todo-title.done span { text-decoration: line-through; color: #9ca3af; }
.empty { margin-top: 24px; text-align: center; color: #6b7280; }
.footer { display: flex; justify-content: space-between; align-items: center; margin-top: 16px; }
.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
```

src/index.css

```
*{
  margin:0;
  padding:0;
  box-sizing:border-box;
  --webkit-font-smoothing:antialiased;
  text-rendering: optimizeLegibility;
}

body{
  padding:10px;
```

```
  background-color:gainsboro;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: empty/short/long/duplicate titles, trimming, case-insensitive search and duplicates, edit keeps its own title, Escape cancels, toggling while filtered, combined search + filter, specific empty states, disabled Clear completed, corrupted or blocked storage.

Trade-offs to mention to the interviewer

- *`useReducer` vs `useState`:* the reducer adds a little boilerplate but puts every update in one pure, testable place. For 2 actions, `useState` would be fine.
- *Derived list on every render vs `useMemo`:* simpler and always correct; add `useMemo` only if profiling shows slow filtering (thousands of todos).
- *No debounce on search:* filtering in memory is instant. Debounce matters when search calls an API.
- *localStorage:* synchronous and limited to about 5 MB per site, and two open tabs overwrite each other (last write wins). Fix it with the `storage` event, or move to a backend.
- *Prop callbacks vs Context:* the tree is only 2 levels deep, so props are clearer. Context helps when the tree gets deep.
- *An empty edit shows an error* instead of deleting the todo (as TodoMVC does): this is safer because the user doesn't lose data by accident.

What to add with more time

- Tests: unit tests for `todoReducer` and `validateTitle` (pure functions); React Testing Library for add → toggle → filter → edit → delete → refresh.
- Optimistic API sync (`POST/PATCH/DELETE /todos`) with rollback on failure and a loading state.
- Drag-and-drop reordering, due dates, priorities, "Toggle all".
- Sync between tabs with the `storage` event; virtualise the list for thousands of todos.
- Keep filter and search in the URL (`?filter=active&q=milk`) so the view can be shared and survives a refresh.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Why reject duplicate todos? Isn't that the user's choice?

It's a product decision, so ask first. I reject them because two identical rows are usually a mistake, and it shows I can validate against existing data. The check ignores case and surrounding spaces.

Q2. Should an empty edit delete the todo (like TodoMVC) or show an error?

Both are valid; I chose an error because deleting on empty loses data by accident. If the interviewer wants TodoMVC behaviour, it's one branch: `if (!draft.trim()) onDelete(id)`.

Q3. Why Save/Cancel buttons and Escape instead of "save on blur"?

Blur fires *before* a click on Cancel, so save-on-blur makes Cancel impossible without extra flags. Explicit buttons + Enter/Escape are predictable and accessible.

State design

Q4. Why `useReducer` instead of `useState` for the todos?

There are 5 kinds of updates (add, toggle, edit, delete, clearCompleted). A reducer puts them in one pure function that's easy to read and test without React, and the components just `dispatch` intent.

Q5. Why not store `filteredTodos` in state?

It would have to be updated in every handler and on every filter/search change; one miss and the list is wrong. Deriving it with `todos.filter(...)` on each render is always correct and cheap.

Q6. Why is the edit mode state inside `TodoItem`, not in `App`?

Only that row cares. Keeping it local means typing in an edit box re-renders only that row, and `App` stays simple. If only one row may be edited at a time, lift `editingId` to `App`.

Q7. Why `completed: boolean` instead of `status: "Active" | "Completed"`?

A boolean can't be misspelled, and toggling is just `!completed`. If more states come later (archived, in progress), switch to an enum.

Hard parts

Q8. Why is `key={index}` a bug here?

Keys tell React which component instance belongs to which item. Delete row 1 and every row shifts index, so row 2's local state (edit mode, draft text) moves to the wrong todo. Use a stable id from the data.

Q9. Why `crypto.randomUUID()` instead of `Date.now()`?

`Date.now()` can return the same value for two todos added in the same millisecond (e.g. a bulk import), which gives duplicate keys. UUIDs are unique; a real backend would assign the id.

Q10. What's wrong with `todo.completed = true; setTodos(todos)`?

It mutates the existing array and object, so React sees the same reference and may skip the re-render; it also breaks memoisation and time-travel debugging. Always return new arrays/objects: `todos.map(t => t.id === id ? { ...t, completed: !t.completed } : t)`.

Q11. How does editing a todo to its own title avoid the "already exists" error?

`validateTitle(title, todos, ignoreId)` skips the todo with `ignoreId` when checking duplicates. Same function for add (no ignoreId) and edit.

Implementation steps

Q12. Why read localStorage with a lazy initialiser?

`useReducer(reducer, undefined, loadTodos)` (or `useState(() => load())`) reads storage once on mount. Calling `load()` directly in the body would read and parse it on every render.

Q13. What if localStorage has corrupted JSON or is blocked?

`JSON.parse` throws and the app goes white. `loadTodos` wraps it in try/catch, checks `Array.isArray`, and falls back to `[]`. `saveTodos` also catches (quota full, private mode).

Q14. How would you test the todo app?

Unit-test the reducer and `validateTitle` (pure functions). Use React Testing Library for add → toggle → filter Active (item disappears) → edit with Escape (unchanged) → delete → reload (persisted, with localStorage mocked).

Performance and scale

Q15. What if there are 10,000 todos?

Filtering is still fast, but rendering 10,000 rows isn't. Virtualise the list (react-window), memoise `TodoItem` with `React.memo` + stable callbacks (`useCallback` / dispatch), and maybe debounce search.

Q16. Two tabs are open. What happens?

Each tab saves its own copy, so the last write wins and the other tab's changes are lost. Fix: listen to the `storage` event and reload todos when another tab writes, or move to a backend.

Q17. How would you sync with a real API?

Optimistic updates: dispatch immediately, send `POST/PATCH/DELETE`, and roll back (with an error toast) if it fails. Use temp ids replaced by server ids, and React Query mutations to manage it.

Trade-offs and "why not X?"

Q18. Why not Context or Redux for the todos?

The tree is two levels deep, so passing props is clearer than a global store. Context helps when many distant components need the todos; Redux / Zustand when the state is large, shared, and needs devtools.

Q19. Why not `useMemo` for `visibleTodos`?

Without measurements it adds complexity for no gain: filtering a few hundred items is microseconds. Add it when profiling shows the filter is slow or when the list is passed to memoised children.

Q20. How would you add drag-and-drop reordering?

Add an `order` field (or keep the array order), a `reorder` reducer action that moves an item from one index to another, and use dnd-kit (accessible, keyboard support) rather than native HTML5 drag events.